

Introduction To Embedding With OpenAI API

Lesson 1: Embedding

- **Embedding Multiple Inputs:** You learned to embed multiple text inputs efficiently by batching them into a single API call. This method reduces the number of API calls and improves performance.
- **Handling Embedding Responses:** You explored how to handle the response from the API when embedding multiple inputs. The response contains a list of dictionaries, each representing the embedding for an input text.
- **Storing Embeddings:** You practiced storing embeddings in a list of dictionaries, associating each embedding with its corresponding text input.
- **Dimensionality Reduction with t-SNE:** You learned to use t-SNE (t-distributed Stochastic Neighbor Embedding) to reduce the high-dimensional embeddings to two dimensions for easier visualization. This helps in understanding how similar texts cluster together based on their semantic meaning.

Here's a code snippet from the lesson:

```
# Set your API key
client = OpenAI(api_key="<OPENAI_API_TOKEN>")

# Extract a list of product short descriptions from products
product_descriptions = [product['short_description'] for product in products]

# Create embeddings for each product description
response = client.embeddings.create(
    model="text-embedding-ada-002",
    input=product_descriptions
)
response_dict = response.model_dump()

# Extract the embeddings from response_dict and store in products
for i, product in enumerate(products):
    product['embedding'] = response_dict['data'][i]['embedding']

print(products[0].items())
```

- **Semantic Search:** You learned how to implement a semantic search engine using embeddings. This involves embedding both the search query and the texts to compare against, computing cosine distances between them, and extracting the texts with the smallest distances.

- **Combining Article Features:** You created a function, `create_article_text`, to combine article features (headline, topic, keywords) into a single string using F-strings and the `join` method.
- **Embedding Text:** You used the OpenAI API to create embeddings for the combined article texts.
- **Finding Closest Matches:** You defined a function, `find_n_closest`, to compute cosine distances between a query vector and a list of embeddings, returning the most similar results.

Here's a code snippet from the lesson:

```
def create_product_text(product):
    return f"""Title: {product['title']}
Description: {product['short_description']}
Category: {product['category']}
Features: {'; '.join(product['features'])}"""

product_texts = [create_product_text(product) for product in products]
product_embeddings = create_embeddings(product_texts)
```

Lesson 2: Embedding for Ai applications

Recommendation systems

- **Recommendation Systems:** These systems recommend items by embedding both the items and the data point, calculating cosine distances, and suggesting the closest items in vector space.
- **Combining Features:** You used the `create_article_text` function to combine article features (headline, topic, keywords) into a single string for embedding.
- **Embedding and Cosine Distances:** You embedded these combined strings using the `create_embeddings` function and calculated cosine distances with the `find_n_closest` function to find the most similar articles.
- **User History:** For more accurate recommendations, you learned to consider multiple data points by embedding the user's history, averaging the embeddings, and filtering out already viewed items.

Here's a snippet demonstrating how to embed and recommend articles:

```
# Combine features and embed articles
current_article_text = create_article_text(current_article)
article_texts = [create_article_text(article) for article in articles]
current_article_embeddings = create_embeddings(current_article_text)[0]
article_embeddings = create_embeddings(article_texts)
```

```
# Find and print the three most similar articles
```

```
hits = find_n_closest(current_article_embeddings, article_embeddings)
```

```
for hit in hits:
```

```
    print(articles[hit["index"]]["headline"])
```

Vector databases for embedding systems (classification)

- **Embedding Class Labels:** You embedded descriptions of class labels (e.g., tech, science, sport, business) to use as reference points for classification.
- **Embedding Articles:** You combined headline and keyword information into a single string to embed and then calculated cosine distances to each class label.
- **Improving Accuracy:** By using detailed class descriptions instead of single words, you improved the model's classification accuracy.

Here's a code snippet for embedding and classifying sentiment in restaurant reviews:

```
# Set your API key
```

```
client = OpenAI(api_key="<OPENAI_API_TOKEN>")
```

```
# Extract and embed the descriptions from sentiments
```

```
class_descriptions = [sentiment["description"] for sentiment in sentiments]
```

```
class_embeddings = create_embeddings(class_descriptions)
```

```
review_embeddings = create_embeddings(reviews)
```

```
def find_closest(query_vector, embeddings):
```

```
    distances = []
```

```
    for index, embedding in enumerate(embeddings):
```

```
        dist = distance.cosine(query_vector, embedding)
```

```
        distances.append({"distance": dist, "index": index})
```

```
    return min(distances, key=lambda x: x["distance"])
```

```
for index, review in enumerate(reviews):
```

```
    closest = find_closest(review_embeddings[index], class_embeddings)
```

```
    label = sentiments[closest["index"]]["label"]
```

```
    print(f"{review}" was classified as {label})
```

Lesson 3: Vector Databases

- **Embedding Class Labels:** You embedded descriptions of class labels (e.g., tech, science, sport, business) to use as reference points for classification.
- **Embedding Articles:** You combined headline and keyword information into a single string to embed and then calculated cosine distances to each class label.

- **Improving Accuracy:** By using detailed class descriptions instead of single words, you improved the model's classification accuracy.

Here's a code snippet for embedding and classifying sentiment in restaurant reviews:

```
# Set your API key
client = OpenAI(api_key="<OPENAI_API_TOKEN>")

# Extract and embed the descriptions from sentiments
class_descriptions = [sentiment['description'] for sentiment in sentiments]
class_embeddings = create_embeddings(class_descriptions)
review_embeddings = create_embeddings(reviews)

def find_closest(query_vector, embeddings):
    distances = []
    for index, embedding in enumerate(embeddings):
        dist = distance.cosine(query_vector, embedding)
        distances.append({"distance": dist, "index": index})
    return min(distances, key=lambda x: x["distance"])

for index, review in enumerate(reviews):
    closest = find_closest(review_embeddings[index], class_embeddings)
    label = sentiments[closest["index"]]["label"]
    print(f"{review} was classified as {label}")
```

Creating Vector Databases With ChromaDB

- **Vector Databases:** Essential for efficiently storing and querying embeddings in production environments.
- **ChromaDB Modes:** ChromaDB can operate in local mode (for development) and client/server mode (for production).

Creating a Persistent Client: This client saves database files to disk, ensuring data persistence.

```
client = chromadb.PersistentClient()
```

Creating Collections: Collections in ChromaDB are similar to tables in relational databases. You created a collection using the OpenAI embedding function.

```
collection = client.create_collection(
    name="netflix_titles",
    embedding_function=OpenAIEmbeddingFunction(api_key="<OPENAI_API_TOKEN>")
)
```

Adding Embeddings: You learned to add documents to the collection, specifying document IDs and texts.

```
collection.add(ids=ids, documents=documents)
```

Estimating Costs: Using the tiktoken library, you calculated the number of tokens in your dataset and estimated the embedding cost.

```
enc = tiktoken.encoding_for_model("text-embedding-ada-002")
total_tokens = sum(len(enc.encode(text)) for text in documents)
cost_per_1k_tokens = 0.0001
print('Total tokens:', total_tokens)
print('Cost:', cost_per_1k_tokens * total_tokens / 1000)
```

- **Inspecting Collections:** Methods like `collection.count()` and `collection.peek()` help you verify the contents of your collection.

Querying and Updating the Database

- **Retrieving a Collection:** You used `client.get_collection()` to retrieve a collection by specifying its name and the embedding function.
- **Querying the Collection:** You learned to query the collection using `collection.query()`, where you pass the query string in a list to the `query_texts` parameter and specify the number of results with `n_results`.
- **Understanding Query Results:** The results are returned as a dictionary containing lists of lists for ids, documents, metadatas, and distances.
- **Updating Items:** You used the `collection.update()` method to update existing items in the collection.
- **Upserting Items:** The `collection.upsert()` method was introduced to either update existing items or add new ones if they don't exist.
- **Deleting Items:** You learned to delete items from the collection using `collection.delete()`.

Here's an example code snippet for querying a collection:

```
# Retrieve the netflix_titles collection
```

```
collection = client.get_collection(
    name="netflix_titles",
    embedding_function=OpenAIEmbeddingFunction(api_key="<OPENAI_API_TOKEN>")
)
```

```
# Query the collection for "films about dogs"
```

```
result = collection.query(
    query_texts=["films about dogs"],
    n_results=3
)
```

```
print(result)
```

Multiple Queries and Filtering

- **Multiple Queries:** You performed multiple queries using embedded texts from reference items. By passing multiple query texts, you retrieved a single dictionary with multiple lists, each corresponding to a query text.
- **Filtering Results:** You filtered out documents that were also in the reference texts during postprocessing to refine your recommendations.
- **Metadata Utilization:** You explored how to use additional metadata (like type and release year) to filter results. This allows for more specific recommendations, such as only movies released after a certain year.

Here's a code snippet demonstrating querying with multiple texts:

```
# Retrieve the netflix_titles collection
collection = client.get_collection(
    name="netflix_titles",
    embedding_function=OpenAIEmbeddingFunction(api_key="<OPENAI_API_TOKEN>")
)

reference_ids = ['s999', 's1000']

# Retrieve the documents for the reference_ids
reference_texts = collection.get(ids=reference_ids)['documents']

# Query using reference_texts
result = collection.query(
    query_texts=reference_texts,
    n_results=3
)

print(result['documents'])
```

Recap

- **Multiple Queries:** You performed multiple queries using embedded texts from reference items. By passing multiple query texts, you retrieved a single dictionary with multiple lists, each corresponding to a query text.
- **Filtering Results:** You filtered out documents that were also in the reference texts during postprocessing to refine your recommendations.
- **Metadata Utilization:** You explored how to use additional metadata (like type and release year) to filter results. This allows for more specific recommendations, such as only movies released after a certain year.

Here's a code snippet demonstrating querying with multiple texts:

```
# Retrieve the netflix_titles collection
collection = client.get_collection(
    name="netflix_titles",
    embedding_function=OpenAIEmbeddingFunction(api_key="<OPENAI_API_TOKEN>")
)

reference_ids = ['s999', 's1000']

# Retrieve the documents for the reference_ids
reference_texts = collection.get(ids=reference_ids)['documents']

# Query using reference_texts
result = collection.query(
    query_texts=reference_texts,
    n_results=3
)

print(result['documents'])
```